

asymptotic notation. For example, you could switch to an efficient serial sort, perhaps quicksort, when the number of elements to be sorted is sufficiently small.

Exercises

26.3-1

Explain how to coarsen the base case of P-MERGE.

26.3-2

Instead of finding a median element in the larger subarray, as P-MERGE does, suppose that the merge procedure finds a median of all the elements in the two sorted subarrays using the result of Exercise 9.3-10. Give pseudocode for an efficient parallel merging procedure that uses this median-finding procedure. Analyze your algorithm.

26.3-3

Give an efficient parallel algorithm for partitioning an array around a pivot, as is done by the PARTITION procedure on page 184. You need not partition the array in place. Make your algorithm as parallel as possible. Analyze your algorithm. (*Hint:* You might need an auxiliary array and might need to make more than one pass over the input elements.)

26.3-4

Give a parallel version of FFT on page 890. Make your implementation as parallel as possible. Analyze your algorithm.

★ 26.3-5

Show how to parallelize SELECT from Section 9.3. Make your implementation as parallel as possible. Analyze your algorithm.

Problems

26-1 *Implementing parallel loops using recursive spawning*

Consider the parallel procedure SUM-ARRAYS for performing pairwise addition on n -element arrays $A[1 : n]$ and $B[1 : n]$, storing the sums in $C[1 : n]$.

SUM-ARRAYS (A, B, C, n)

```
1 parallel for  $i = 1$  to  $n$ 
2    $C[i] = A[i] + B[i]$ 
```

- a.** Rewrite the parallel loop in SUM-ARRAYS using recursive spawning in the manner of P-MAT-VEC-RECURSIVE. Analyze the parallelism.

Consider another implementation of the parallel loop in SUM-ARRAYS given by the procedure SUM-ARRAYS', where the value *grain-size* must be specified.

SUM-ARRAYS'(A, B, C, n)

```
1  $grain-size = ?$  // to be determined
2  $r = \lceil n/grain-size \rceil$ 
3 for  $k = 0$  to  $r - 1$ 
4   spawn ADD-SUBARRAY ( $A, B, C, k \cdot grain-size + 1,$ 
                         $\min \{(k + 1) \cdot grain-size, n\}$ )
5 sync
```

ADD-SUBARRAY (A, B, C, i, j)

```
1 for  $k = i$  to  $j$ 
2    $C[k] = A[k] + B[k]$ 
```

- b.** Suppose that you set $grain-size = 1$. What is the resulting parallelism?
- c.** Give a formula for the span of SUM-ARRAYS' in terms of n and $grain-size$. Derive the best value for $grain-size$ to maximize parallelism.

26-2 Avoiding a temporary matrix in recursive matrix multiplication

The P-MATRIX-MULTIPLY-RECURSIVE procedure on page 772 must allocate a temporary matrix D of size $n \times n$, which can adversely affect the constants hidden by the Θ -notation. The procedure has high parallelism, however: $\Theta(n^3/\log^2 n)$. For example, ignoring the constants in the Θ -notation, the parallelism for multiplying 1000×1000 matrices comes to approximately $1000^3/10^2 = 10^7$, since $\lg 1000 \approx 10$. Most parallel computers have far fewer than 10 million processors.

- a.** Parallelize MATRIX-MULTIPLY-RECURSIVE without using temporary matrices so that it retains its $\Theta(n^3)$ work. (*Hint:* Spawn the recursive calls, but insert a **sync** in a judicious location to avoid races.)
- b.** Give and solve recurrences for the work and span of your implementation.
- c.** Analyze the parallelism of your implementation. Ignoring the constants in the Θ -notation, estimate the parallelism on 1000×1000 matrices. Compare with the parallelism of P-MATRIX-MULTIPLY-RECURSIVE, and discuss whether the trade-off would be worthwhile.

26-3 *Parallel matrix algorithms*

Before attempting this problem, it may be helpful to read Chapter 28.

- a.** Parallelize the LU-DECOMPOSITION procedure on page 827 by giving pseudocode for a parallel version of this algorithm. Make your implementation as parallel as possible, and analyze its work, span, and parallelism.
- b.** Do the same for LUP-DECOMPOSITION on page 830.
- c.** Do the same for LUP-SOLVE on page 824.
- d.** Using equation (28.14) on page 835, write pseudocode for a parallel algorithm to invert a symmetric positive-definite matrix. Make your implementation as parallel as possible, and analyze its work, span, and parallelism.

26-4 Parallel reductions and scan (prefix) computations

A $\otimes$ -*reduction* of an array $x[1 : n]$, where $\otimes$ is an associative operator, is the value $y = x[1] \otimes x[2] \otimes \cdots \otimes x[n]$. The REDUCE procedure computes the $\otimes$ -reduction of a subarray $x[i : j]$ serially.

```
REDUCE ( $x, i, j$ )
```

```
1  $y = x[i]$ 
```

```
2 for  $k = i + 1$  to  $j$ 
```

```
3    $y = y \otimes x[k]$ 
```

```
4 return  $y$ 
```

- a. Design and analyze a parallel algorithm P-REDUCE that uses recursive spawning to perform the same function with $\Theta(n)$ work and $\Theta(\lg n)$ span.

A related problem is that of computing a $\otimes$ -*scan*, sometimes called a $\otimes$ -*prefix computation*, on an array $x[1 : n]$, where $\otimes$ is once again an associative operator. The $\otimes$ -scan, implemented by the serial procedure SCAN, produces the array $y[1 : n]$ given by

$$y[1] = x[1],$$

$$y[2] = x[1] \otimes x[2],$$

$$y[3] = x[1] \otimes x[2] \otimes x[3],$$

$$\vdots$$

$$y[n] = x[1] \otimes x[2] \otimes x[3] \otimes \cdots \otimes x[n],$$

that is, all prefixes of the array x “summed” using the $\otimes$ operator.

```
SCAN ( $x, n$ )
```

```
1 let  $y[1 : n]$  be a new array
```

```
2  $y[1] = x[1]$ 
```

```
3 for  $i = 2$  to  $n$ 
```

```
4    $y[i] = y[i - 1] \otimes x[i]$ 
```

```
5 return  $y$ 
```

Parallelizing SCAN is not straightforward. For example, simply changing the **for** loop to a **parallel for** loop would create races, since each iteration of the loop body depends on the previous iteration. The procedures P-SCAN-1 and P-SCAN-1-AUX perform the $\otimes$ -scan in parallel, albeit inefficiently.

```

P-SCAN-1( $x, n$ )
1 let  $y[1] : n$  be a new array
2 P-SCAN-1-AUX ( $x, y, 1, n$ )
3 return  $y$ 

P-SCAN-1-AUX ( $x, y, i, j$ )
1 parallel for  $l = i$  to  $j$ 
2    $y[l] = \text{P-REDUCE}(x, 1, l)$ 

```

b. Analyze the work, span, and parallelism of P-SCAN-1.

The procedures P-SCAN-2 and P-SCAN-2-AUX use recursive spawning to perform a more efficient $\otimes$ -scan.

```

P-SCAN-2( $x, n$ )
1 let  $y[1] : n$  be a new array
2 P-SCAN-2-AUX ( $x, y, 1, n$ )
3 return  $y$ 

P-SCAN-2-AUX ( $x, y, i, j$ )
1 if  $i == j$ 
2    $y[i] = x[i]$ 
3 else  $k = \lfloor (i + j)/2 \rfloor$ 
4   spawn P-SCAN-2-AUX ( $x, y, i, k$ )
5   P-SCAN-2-AUX ( $x, y, k + 1, j$ )
6   sync
7   parallel for  $l = k + 1$  to  $j$ 
8      $y[l] = y[k] \otimes y[l]$ 

```

- c. Argue that P-SCAN-2 is correct, and analyze its work, span, and parallelism.

To improve on both P-SCAN-1 and P-SCAN-2, perform the $\otimes$ -scan in two distinct passes over the data. The first pass gathers the terms for various contiguous subarrays of x into a temporary array t , and the second pass uses the terms in t to compute the final result y . The pseudocode in the procedures P-SCAN-3, P-SCAN-UP, and P-SCAN-DOWN on the facing page implements this strategy, but certain expressions have been omitted.

- d. Fill in the three missing expressions in line 8 of P-SCAN-UP and lines 5 and 6 of P-SCAN-DOWN. Argue that with the expressions you supplied, P-SCAN-3 is correct. (*Hint*: Prove that the value v passed to P-SCAN-DOWN(v, x, t, y, i, j) satisfies $v = x[1] \otimes x[2] \otimes \cdots \otimes x[i - 1]$.)

- e. Analyze the work, span, and parallelism of P-SCAN-3.

- f. Describe how to rewrite P-SCAN-3 so that it doesn't require the use of the temporary array t .

- ★ g. Give an algorithm P-SCAN-4(x, n) for a scan that operates in place. It should place its output in x and require only constant auxiliary storage.

- h. Describe an efficient parallel algorithm that uses a $+$ -scan to determine whether a string of parentheses is well formed. For example, the string $((())())$ is well formed, but the string $((()))(())$ is not. (*Hint*: Interpret $($ as a 1 and $)$ as a -1 , and then perform a $+$ -scan.)

P-SCAN-3(x, n)

```
1 let  $y[1] : n$  and  $t[1 : n]$  be new arrays
2  $y[1] = x[1]$ 
3 if  $n > 1$ 
4   P-SCAN-UP( $x, t, 2, n$ )
5   P-SCAN-DOWN( $x[1], x, t, y, 2, n$ )
```

```
6 return y
```

```
P-SCAN-UP (x, t, i, j)
```

```
1 if i == j
```

```
2   return x[i]
```

```
3 else
```

```
4   k =  $\lfloor (i + j)/2 \rfloor$ 
```

```
5   t[k] = spawn P-SCAN-UP (x, t, i, k)
```

```
6   right = P-SCAN-UP (x, t, k + 1, j)
```

```
7   sync
```

```
8   return _____ // fill in the blank
```

```
P-SCAN-DOWN (v, x, t, y, i, j)
```

```
1 if i == j
```

```
2   y[i] = v  $\otimes$  x[i]
```

```
3 else
```

```
4   k =  $\lfloor (i + j)/2 \rfloor$ 
```

```
5   spawn P-SCAN-DOWN (_____, x, t, y, i, // fill in the  
      k) blank
```

```
6   P-SCAN-DOWN (_____, x, t, y, k + 1, j) // fill in the  
      blank
```

```
7   sync
```

26-5 *Parallelizing a simple stencil calculation*

Computational science is replete with algorithms that require the entries of an array to be filled in with values that depend on the values of certain already computed neighboring entries, along with other information that does not change over the course of the computation. The pattern of neighboring entries does not change during the computation and is called a *stencil*. For example, Section 14.4 presents a stencil algorithm to compute a longest common subsequence, where the value in entry $c[i, j]$ depends only on the values in $c[i - 1, j]$, $c[i, j - 1]$, and $c[i - 1, j - 1]$, as well as the elements x_i and y_j within the two sequences given as inputs. The input sequences are fixed, but the

algorithm fills in the two-dimensional array c so that it computes entry $c[i, j]$ after computing all three entries $c[i - 1, j]$, $c[i, j - 1]$, and $c[i - 1, j - 1]$.

This problem examines how to use recursive spawning to parallelize a simple stencil calculation on an $n \times n$ array A in which the value placed into entry $A[i, j]$ depends only on values in $A[i', j']$, where $i' \leq i$ and $j' \leq j$ (and of course, $i' \neq i$ or $j' \neq j$). In other words, the value in an entry depends only on values in entries that are above it and/or to its left, along with static information outside of the array. Furthermore, we assume throughout this problem that once the entries upon which $A[i, j]$ depends have been filled in, the entry $A[i, j]$ can be computed in $\Theta(1)$ time (as in the LCS-LENGTH procedure of Section 14.4).

Partition the $n \times n$ array A into four $n/2 \times n/2$ subarrays as follows:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}. \quad (26.9)$$

You can immediately fill in subarray A_{11} recursively, since it does not depend on the entries in the other three subarrays. Once the computation of A_{11} finishes, you can fill in A_{12} and A_{21} recursively in parallel, because although they both depend on A_{11} , they do not depend on each other. Finally, you can fill in A_{22} recursively.

- a.** Give parallel pseudocode that performs this simple stencil calculation using a divide-and-conquer algorithm SIMPLE-STENCIL based on the decomposition (26.9) and the discussion above. (Don't worry about the details of the base case, which depends on the specific stencil.) Give and solve recurrences for the work and span of this algorithm in terms of n . What is the parallelism?
- b.** Modify your solution to part (a) to divide an $n \times n$ array into nine $n/3 \times n/3$ subarrays, again recursing with as much parallelism as possible. Analyze this algorithm. How much more or less parallelism does this algorithm have compared with the algorithm from part (a)?
- c.** Generalize your solutions to parts (a) and (b) as follows. Choose an integer $b \geq 2$. Divide an $n \times n$ array into b^2 subarrays, each of size n/b

$\times n/b$, recursing with as much parallelism as possible. In terms of n and b , what are the work, span, and parallelism of your algorithm? Argue that, using this approach, the parallelism must be $o(n)$ for any choice of $b \geq 2$. (*Hint:* For this argument, show that the exponent of n in the parallelism is strictly less than 1 for any choice of $b \geq 2$.)

- d.** Give pseudocode for a parallel algorithm for this simple stencil calculation that achieves $\Theta(n/\lg n)$ parallelism. Argue using notions of work and span that the problem has $\Theta(n)$ inherent parallelism. Unfortunately, simple fork-join parallelism does not let you achieve this maximal parallelism.

26-6 *Randomized parallel algorithms*

Like serial algorithms, parallel algorithms can employ random-number generators. This problem explores how to adapt the measures of work, span, and parallelism to handle the expected behavior of randomized task-parallel algorithms. It also asks you to design and analyze a parallel algorithm for randomized quicksort.

- a.** Explain how to modify the work law (26.2), span law (26.3), and greedy scheduler bound (26.4) to work with expectations when T_P , T_1 , and T_∞ are all random variables.
- b.** Consider a randomized parallel algorithm for which 1% of the time, $T_1 = 10^4$ and $T_{10,000} = 1$, but for the remaining 99% of the time, $T_1 = T_{10,000} = 10^9$. Argue that the *speedup* of a randomized parallel algorithm should be defined as $E[T_1]/E[T_P]$, rather than $E[T_1/T_P]$.
- c.** Argue that the *parallelism* of a randomized task-parallel algorithm should be defined as the ratio $E[T_1]/E[T_\infty]$.
- d.** Parallelize the RANDOMIZED-QUICKSORT algorithm on page 192 by using recursive spawning to produce P-RANDOMIZED-QUICKSORT. (Do not parallelize RANDOMIZED-PARTITION.)
- e.** Analyze your parallel algorithm for randomized quicksort. (*Hint:* Review the analysis of RANDOMIZED-SELECT on page 230.)

f. Parallelize RANDOMIZED-SELECT on page 230. Make your implementation as parallel as possible. Analyze your algorithm. (*Hint:* Use the partitioning algorithm from Exercise 26.3-3.)

Chapter notes

Parallel computers and algorithmic models for parallel programming have been around in various forms for years. Prior editions of this book included material on sorting networks and the PRAM (Parallel Random-Access Machine) model. The data-parallel model [58, 217] is another popular algorithmic programming model, which features operations on vectors and matrices as primitives. The notion of sequential consistency is due to Lamport [275].

Graham [197] and Brent [71] showed that there exist schedulers achieving the bound of Theorem 26.1. Eager, Zahorjan, and Lazowska [129] showed that any greedy scheduler achieves this bound and proposed the methodology of using work and span (although not by those names) to analyze parallel algorithms. Blelloch [57] developed an algorithmic programming model based on work and span (which he called “depth”) for data-parallel programming. Blumofe and Leiserson [63] gave a distributed scheduling algorithm for task-parallel computations based on randomized “work-stealing” and showed that it achieves the bound $E[T_P] \leq T_1/P + O(T_\infty)$. Arora, Blumofe, and Plaxton [20] and Blelloch, Gibbons, and Matias [61] also provided provably good algorithms for scheduling task-parallel computations. The recent literature contains many algorithms and strategies for scheduling parallel programs.

The parallel pseudocode and programming model were influenced by Cilk [290, 291, 383, 396]. The open-source project OpenCilk (www.opencilk.org) provides Cilk programming as an extension to the C and C++ programming languages. All of the parallel algorithms in this chapter can be coded straightforwardly in Cilk.

Concerns about nondeterministic parallel programs were expressed by Lee [281] and Bocchino, Adve, Adve, and Snir [64]. The algorithms literature contains many algorithmic strategies (see, for example, [60, 85,

118, 140, 160, 282, 283, 412, 461]) for detecting races and extending the fork-join model to avoid or safely embrace various kinds of nondeterminism. Blleloch, Fineman, Gibbons, and Shun [59] showed that deterministic parallel algorithms can often be as fast as, or even faster than, their nondeterministic counterparts.

Several of the parallel algorithms in this chapter appeared in unpublished lecture notes by C. E. Leiserson and H. Prokop and were originally implemented in Cilk. The parallel merge-sorting algorithm was inspired by an algorithm due to Akl [12].

¹ In mathematics, a projection is an idempotent function, that is, a function f such that $f \circ f = f$. In this case, the function f maps the set $\mathcal{P}$ of fork-join programs to the set $\mathcal{P}_S \subset \mathcal{P}$ of serial programs, which are themselves fork-join programs with no parallelism. For a fork-join program $x \in \mathcal{P}$, since we have $f(f(x)) = f(x)$, the serial projection, as we have defined it, is indeed a mathematical projection.

² Also called a *computation dag* in the literature.